Python Engineering - Functions

validate_inputs(), booleans, and the difference between defining and calling a function

Support note for **week_01_logic -> class_04_functions**. This document summarizes the bug found in EX12 and why returning **True / False** is a natural design for a validation function.

1. A function is also an object

When Python reads a **def** statement, it creates a function object and stores it under a name. The function body is not executed at that moment.

```
def validate_inputs(h0, v0, t0, g, dt):
    ...

# This does NOT execute the function:
validate_inputs

# This DOES execute it:
validate_inputs(h0, v0, t0, g, dt)
```

Key idea: `validate_inputs` is the function object. `validate_inputs(...)` is a call that executes the body and returns a value.

2. Why the first if did not work

```
if validate_inputs is None:
    print('Invalid inputs')
```

That condition asks whether the function object itself is None. It never calls the function, so h0, v0, t0, g, and dt are never validated.

3. The second issue: None for both valid and invalid

The first version returned **None** for invalid data, but if all inputs were valid the function reached the end without a return. In Python, reaching the end of a function also returns None.

Case	First-version result
Invalid input	None
Valid input	None

4. Natural solution: return a boolean

```
def validate_inputs(h0, v0, t0, g, dt):
    if h0 < 0:
        return False
    if v0 <= 0:
        return False
    if t0 < 0:
        return False
    if dt <= 0:
        return False
    if g <= 0:
        return False
    return True
```

The function now answers a binary question: **are the inputs valid?** That is why True/False expresses the intent more clearly than None.

```
if validate_inputs(h0, v0, t0, g, dt):
    # Run simulation
    ...
```

It reads almost like English: **if validate inputs, run the simulation**.

5. What happens during the call

Python binds the arguments to the parameters, creates the local scope for that call, evaluates the checks in order, and immediately ends the call when it reaches a return.

```
CALL
  -> bind arguments
  -> create local scope
  -> evaluate validations
  -> return False OR True
  -> continue in main
```

6. Guard clauses

Because every invalid case immediately returns False, elif is unnecessary. If Python reaches the next if statement, the previous check did not terminate the function.

Pattern: reject invalid cases early and keep the main path clean. This style is commonly called *guard clauses*.

7. Connection to EX12

The EX12 flow becomes: validate -> simulate -> analytical solution -> errors -> display. Validation is no longer just a print; it controls whether the rest of the program is allowed to run.

Learning tracker

Concept	Status
Function object vs function call	Learned
Implicit None return	Learned
Boolean return	Learned
Guard clauses	Introduced
Flow control from main	Learned